

# طراحی الگوریتم‌ها – میانه و مرتبه‌های آماری

Introduction to Algorithm

مهدی جوانمردی

# مرور جلسه قبل

## مرتب‌سازی سطلی BUCKET SORT

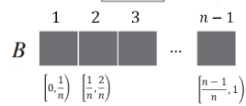
$n$ -element array  $A$

$$0 \leq A[i] < 1$$

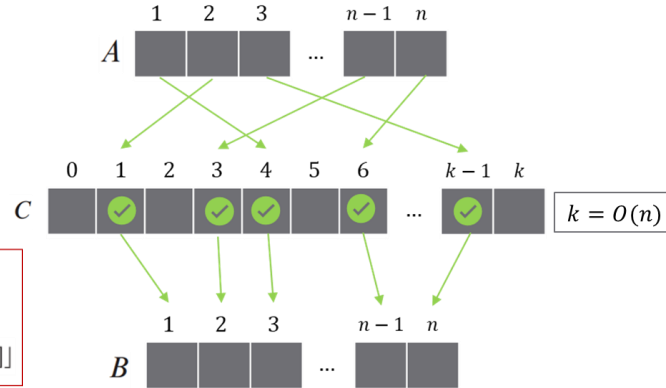
توزیع نرمال در بازه  $[0,1)$

$B[0 \dots n-1]$

سطل



$A[i]$   
 $\downarrow$   
 $[nA[i]]$



## مرتب‌سازی خطی RADIX SORT

اطلاعات مازاد: اعداد دارای  $d$  رقم و هر رقم  $k$  مقدار متفاوت

|     |     |     |     |                                                    |
|-----|-----|-----|-----|----------------------------------------------------|
| 329 | 720 | 720 | 329 | RADIX-SORT( $A, d$ )                               |
| 457 | 355 | 329 | 355 | 1 for $i = 1$ to $d$                               |
| 657 | 436 | 436 | 436 | 2 use a stable sort to sort array $A$ on digit $i$ |
| 839 | 457 | 839 | 457 |                                                    |
| 436 | 657 | 355 | 657 |                                                    |
| 720 | 329 | 457 | 720 |                                                    |
| 355 | 839 | 657 | 839 |                                                    |

• زمان اجرا:  $\theta(d(n+k))$

• اگر  $k = O(n)$  و  $d = O(1)$  باشد زمان اجرای radix sort  $\theta(n)$  خواهد بود

## تحلیل زمانی RADIX SORT

• اگر  $n$  عدد  $b$  بیتی داشته باشیم، برای هر عدد مثبت دلخواه  $r \leq b$  خواهیم داشت:

زمان اجرایی radix sort برای این اعداد  $\theta((b/r)(n + 2^r))$  خواهد بود

به شرطی که مرتب‌سازی پایدار استفاده شده  $\theta(n+k)$  باشد

برای اعداد  $n$  و  $b$  تعیین  $r \leq b$  بگونه‌ای که زمان اجرای  $(b/r)(n + 2^r)$  را کمینه کند

حالت اول:  $b < \lceil \lg n \rceil$  ← برای  $r = b$  خواهیم داشت:  $\theta(n)$

حالت دوم:  $b \geq \lceil \lg n \rceil$  ← برای  $r = \lceil \lg n \rceil$  خواهیم داشت:  $\theta(bn/\lg n)$

در صورتی که  $b = O(\lg n)$  باشد با انتخاب  $r = \lceil \lg n \rceil$  زمان اجرای radix sort برابر  $O(n)$

## تحلیل زمانی BUCKET SORT

BUCKET-SORT( $A$ )

- 1 let  $B[0 \dots n-1]$  be a new array
- 2  $n = A.length$
- 3 for  $i = 0$  to  $n-1$
- 4 make  $B[i]$  an empty list
- 5 for  $i = 1$  to  $n$
- 6 insert  $A[i]$  into list  $B[\lfloor nA[i] \rfloor]$
- 7 for  $i = 0$  to  $n-1$
- 8 sort list  $B[i]$  with insertion sort
- 9 concatenate the lists  $B[0], B[1], \dots, B[n-1]$  together in order

$$T(n) = \Theta(n) + \sum_{i=0}^{n-1} O(n_i^2)$$

$n_i$  متغیر تصادفی نشانگر تعداد المان‌ها در سطل  $B_i$

$$= \Theta(n) + \sum_{i=0}^{n-1} O\left(2 - \frac{1}{n}\right) = \Theta(n)$$

# فصل ۹ کتاب

- میانۀ و مرتبه‌های آماری
  - کمینه و بیشینه
  - امید زمان اجرای الگوریتم انتخاب
  - بدترین زمان اجرای الگوریتم انتخاب

|          |                                     |            |
|----------|-------------------------------------|------------|
| <b>9</b> | <b>Medians and Order Statistics</b> | <b>213</b> |
| 9.1      | Minimum and maximum                 | 214        |
| 9.2      | Selection in expected linear time   | 215        |
| 9.3      | Selection in worst-case linear time | 220        |

# مرتبه‌های آماری

- انتخاب  $i$  امین کوچکترین عنصر در یک آرایه عددی  $n$  عنصری
- کمینه اعداد: اولین مرتبه آماری
- بیشینه اعداد:  $n$  امین مرتبه آماری
- میانه اعداد: عدد وسطی آرایه
  - اگر  $n$  فرد باشد: عنصر  $(n + 1)/2$  ام
  - اگر  $n$  زوج باشد: عنصرهای  $n/2$  و  $n/2 + 1$
  - حالت کلی: میانه پایین  $\lfloor (n + 1)/2 \rfloor$  و میانه بالا  $\lceil (n + 1)/2 \rceil$
- میانه در این درس برای سادگی میانه پایین فرض می‌شود
- اعداد آرایه یکتا هستند: هیچ دو عدد تکراری نداریم (صرفاً برای راحتی، قابل تعمیم به حالت کلی)

# مسئله انتخاب Selection Problem

**Input:** A set  $A$  of  $n$  (distinct) numbers and an integer  $i$ , with  $1 \leq i \leq n$ .

**Output:** The element  $x \in A$  that is larger than exactly  $i - 1$  other elements of  $A$ .

- ورودی: مجموعه  $A$  با  $n$  عدد یکتا و یک عدد طبیعی  $i$  به شرط  $1 \leq i \leq n$
- خروجی: عنصر  $x \in A$  به شرطی که  $x$  دقیقاً از  $i - 1$  عنصر دیگر مجموعه  $A$  بزرگتر باشد
- راحل حل ابتدایی  $O(n \lg n)$ : مرتب‌سازی آرایه با  $O(n \lg n)$  و انتخاب  $i$  امین عنصر آرایه با  $O(1)$
- هدف این بخش از درس: ارائه الگوریتم سریع‌تر برای مسئله انتخاب

# انتخاب کمینه یا بیشینه

- چه تعداد مقایسه برای تعیین **کمینه** یا **بیشینه** مورد نیاز است؟
- حد بالا: شروع از اولین عنصر و انجام  $n - 1$  مقایسه و نگهداری کمینه یا بیشینه

MINIMUM( $A$ )

```

1   $min = A[1]$ 
2  for  $i = 2$  to  $A.length$ 
3      if  $min > A[i]$ 
4           $min = A[i]$ 
5  return  $min$ 
```

- آیا این بهترین کار است؟

- جواب: بله! می‌توان نشان داد که  $n - 1$  مقایسه حد پایین هم هست

- تورنمنت اعداد: هرکسی به غیر از مینیمم حداقل یک باخت!



# انتخاب کمینه و بیشینه بصورت همزمان

• در برخی موارد نیاز به پیدا کردن همزمان ماکزیمم و مینیمم داریم

• مثال: نرمال کردن مجموعه ای از اعداد

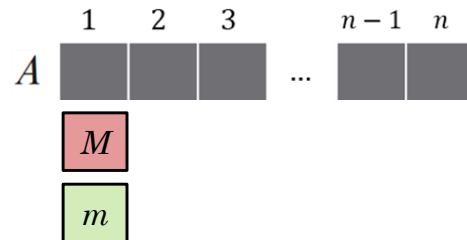
## Normalization Formula

$$X_{normalized} = \frac{(X - X_{minimum})}{(X_{maximum} - X_{minimum})}$$



asymptotically optimal

• جواب  $\Theta(n)$ : تعیین مستقل مینیمم و ماکزیمم هرکدام  $n - 1$  مقایسه، مجموعاً  $2n - 2$  مقایسه

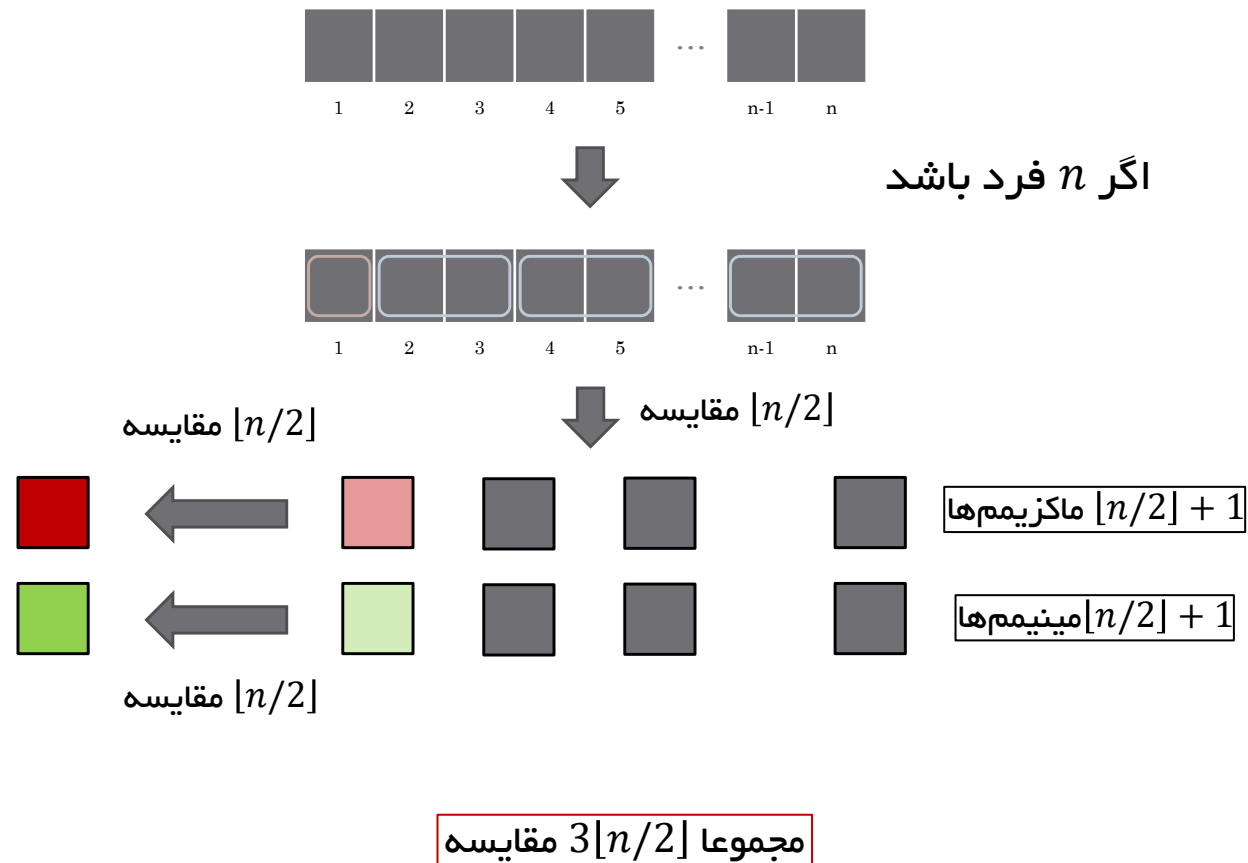


• روش بهتر؟ جواب: بله! ← تعیین همزمان مینیمم و ماکزیمم با حداکثر  $3\lfloor n/2 \rfloor$  مقایسه

• به جای مقایسه هر عنصر با مینیمم و ماکزیمم فعلی ← مقایسه عناصر بصورت جفت

# انتخاب کمینه و بیشینه بصورت همزمان

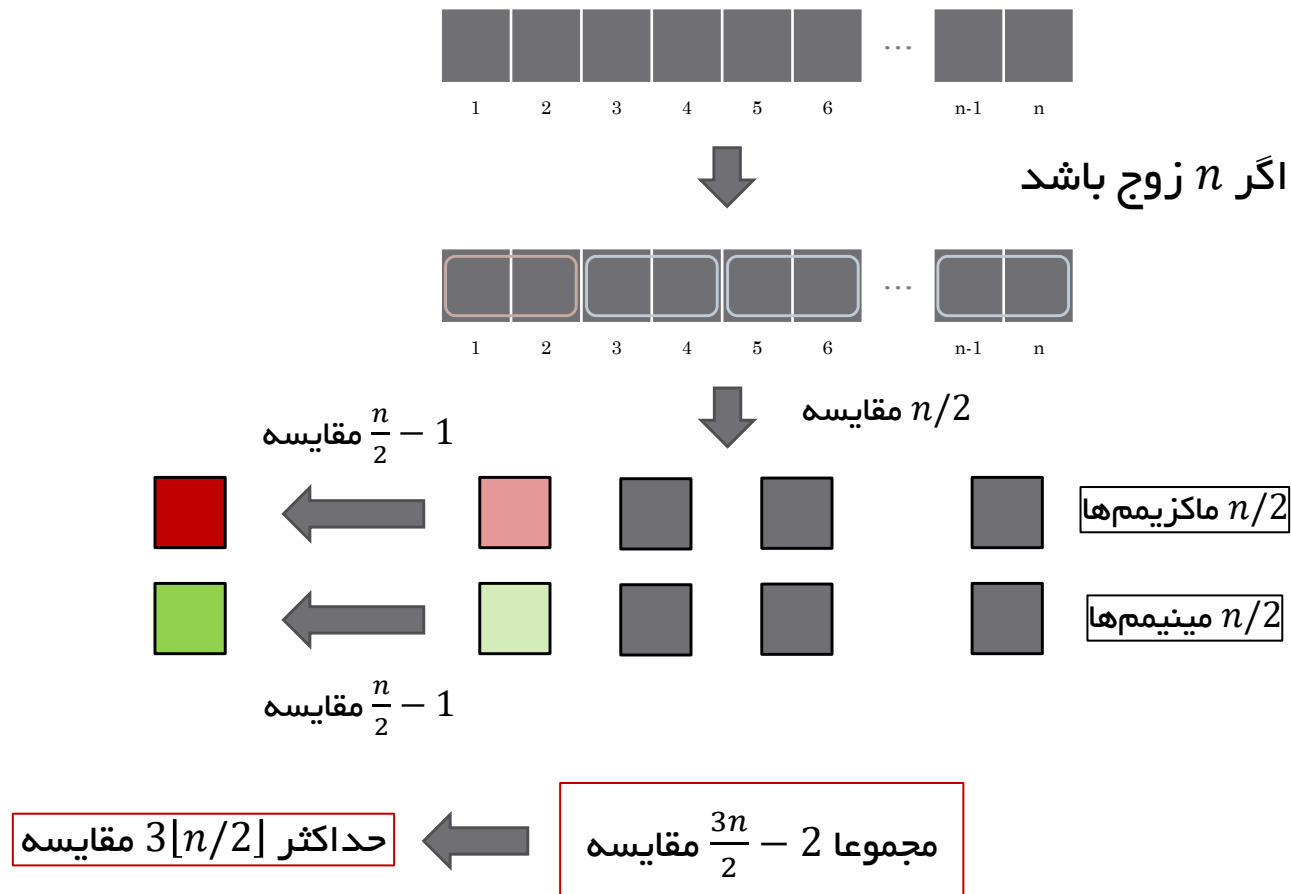
- به جای مقایسه هر عنصر با مینیمم و ماکزیمم فعلی ← مقایسه عناصر بصورت جفت





# انتخاب کمینه و بیشینه بصورت همزمان

- به جای مقایسه هر عنصر با مینیمم و ماکزیمم فعلی ← مقایسه عناصر بصورت جفت



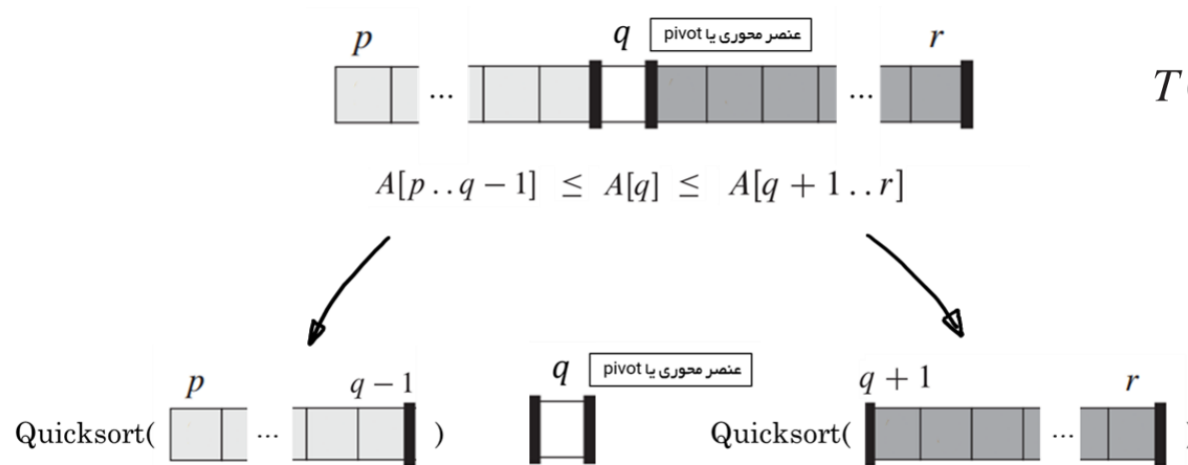
# انتخاب کمینه و بیشینه بصورت همزمان

## 9.1-2 ★

Prove the lower bound of  $\lceil 3n/2 \rceil - 2$  comparisons in the worst case to find both the maximum and minimum of  $n$  numbers. (*Hint: Consider how many numbers are potentially either the maximum or minimum, and investigate how a comparison affects these counts.*)

# مسئله انتخاب با زمان اجرای متوسط خطی

- ورودی: مجموعه  $A$  با  $n$  عدد یکتا و یک عدد طبیعی  $i$  به شرط  $1 \leq i \leq n$
- خروجی: عنصر  $x \in A$  به شرطی که  $x$  دقیقا از  $i - 1$  عنصر دیگر مجموعه  $A$  بزرگتر باشد
- راه حل با زمان متوسط  $\Theta(n)$ : روش تقسیم و حل مشابه ایده اصلی مرتب‌سازی سریع



$$T(n) = (T(q) + T(n - q - 1)) + \Theta(n)$$

Expected Running Time:  $\Theta(n \lg n)$

Expected Running Time:  $\Theta(n)$

دارای  $q$  عنصر

دارای  $n - q - 1$  عنصر

# مسئله انتخاب با زمان اجرای متوسط خطی

- ورودی: مجموعه  $A$  با  $n$  عدد یکتا و یک عدد طبیعی  $i$  به شرط  $1 \leq i \leq n$
- خروجی: عنصر  $x \in A$  به شرطی که  $x$  دقیقاً از  $i - 1$  عنصر دیگر مجموعه  $A$  بزرگتر باشد

RANDOMIZED-SELECT( $A, p, r, i$ )

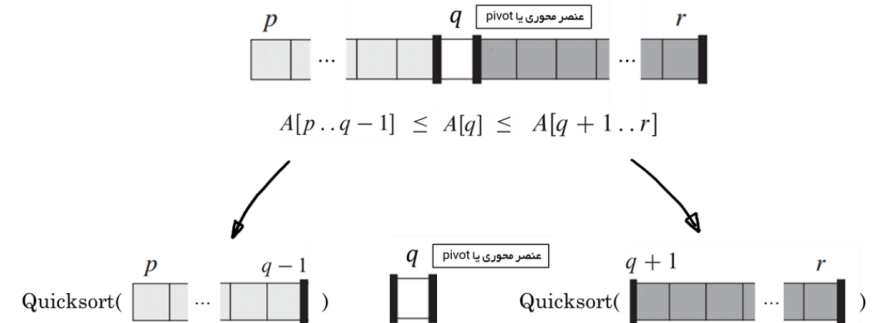
```

1  if  $p == r$ 
2      return  $A[p]$ 
3   $q = \text{RANDOMIZED-PARTITION}(A, p, r)$ 
4   $k = q - p + 1$ 
5  if  $i == k$            // the pivot value is the answer
6      return  $A[q]$ 
7  elseif  $i < k$ 
8      return RANDOMIZED-SELECT( $A, p, q - 1, i$ )
9  else return RANDOMIZED-SELECT( $A, q + 1, r, i - k$ )
    
```

RANDOMIZED-PARTITION( $A, p, r$ )

```

1   $i = \text{RANDOM}(p, r)$ 
2  exchange  $A[r]$  with  $A[i]$ 
3  return PARTITION( $A, p, r$ )
    
```



دارای  $q$  عنصر

دارای  $n - q - 1$  عنصر

$$K = q - p - 1 \leq i$$

در نیمه دوم دنبال  $i$ -امین میگردیم

$$K = q - p - 1 > i$$

در نیمه اول دنبال  $i$ -امین میگردیم

$$K = q - p - 1 = i$$

pivot برابر  $i$ -امین عنصر است

# تحلیل زمانی RANDOMIZED-SELECT

• بدترین زمان اجرا حتی برای پیدا کردن مینیمم:  $\Theta(n^2)$

• زمان اجرای متوسط:  $\Theta(n)$

• زمان اجرای تعیین ا امین عنصر از آرایه  $A[p..r]$  یک متغیر تصادفی:  $T(n)$

• هدف: محاسبه  $E[T(n)]$

• استفاده از Randomized-partition:

• احتمال اینکه آرایه  $A[p..q]$  برای همه مقادیر  $k$  در محدوده  $1 \leq k \leq n$  دارای  $k$  عنصر باشد برابر با  $1/n$

$$X_k = I \{ \text{the subarray } A[p..q] \text{ has exactly } k \text{ elements} \}$$

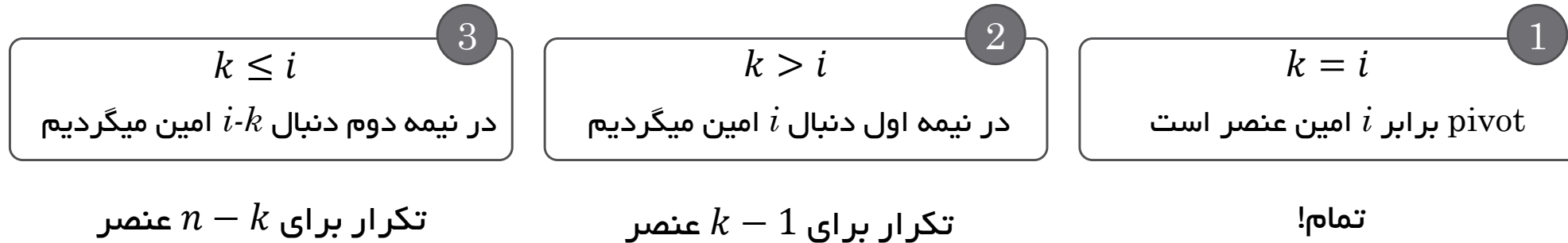
$$E[X_k] = 1/n$$

آمار و احتمالات

$$I(A) = \begin{cases} 1, & \text{if } A \text{ happen} \\ 0, & \text{if } A \text{ not happen} \end{cases}$$

متغیر تصادفی شاخص

# تحلیل زمانی RANDOMIZED-SELECT



$$\begin{aligned}
 T(n) &\leq \sum_{k=1}^n X_k \cdot (T(\max(k-1, n-k)) + O(n)) \\
 &= \sum_{k=1}^n X_k \cdot T(\max(k-1, n-k)) + O(n) .
 \end{aligned}$$

# تحلیل زمانی RANDOMIZED-SELECT

• محاسبه امید ریاضی زمان اجرا یا زمان اجرای متوسط:

$$E[T(n)]$$

$$\leq E \left[ \sum_{k=1}^n X_k \cdot T(\max(k-1, n-k)) + O(n) \right]$$

$$= \sum_{k=1}^n E[X_k \cdot T(\max(k-1, n-k))] + O(n)$$

ویژگی خطی بودن

$$= \sum_{k=1}^n E[X_k] \cdot E[T(\max(k-1, n-k))] + O(n)$$

ضرب متغیر مستقل

$$= \sum_{k=1}^n \frac{1}{n} \cdot E[T(\max(k-1, n-k))] + O(n)$$

امید ریاضی و برخی خواص آن

$$E(X) = \sum_{i=1}^n p_i x_i$$

تعریف امید ریاضی برای متغیر تصادفی  $x_i$

$$E(aX + b) = aE(X) + b$$

ویژگی خطی بودن امید ریاضی

$$E(XY) = E(X)E(Y)$$

برای متغیر تصادفی مستقل  $X$  و  $Y$  داریم:

# تحلیل زمانی RANDOMIZED-SELECT

• ادامه محاسبات:

$$E[T(n)] = \sum_{k=1}^n \frac{1}{n} \cdot E[T(\max(k-1, n-k))] + O(n)$$

$$\max(k-1, n-k) = \begin{cases} k-1 & \text{if } k > \lceil n/2 \rceil, \\ n-k & \text{if } k \leq \lceil n/2 \rceil. \end{cases}$$

|       |       |       |       |       |       |       |       |       |       |
|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
|       |       |       |       |       |       |       |       |       |       |
| 1     | 2     | 3     | 4     | 5     | 6     | 7     | 8     | 9     | 10    |
| $n-k$ | $n-k$ | $n-k$ | $n-k$ | $n-k$ | $k-1$ | $k-1$ | $k-1$ | $k-1$ | $k-1$ |
| 9     | 8     | 7     | 6     | 5     | 5     | 6     | 7     | 8     | 9     |

If  $n$  is even, each term from  $T(\lceil n/2 \rceil)$  up to  $T(n-1)$  appears exactly twice in the summation, and if  $n$  is odd, all these terms appear twice and  $T(\lfloor n/2 \rfloor)$  appears once. Thus, we have

$$E[T(n)] \leq \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[T(k)] + O(n)$$



# تحلیل زمانی RANDOMIZED-SELECT

$$E[T(n)] \leq \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} E[T(k)] + O(n) \longrightarrow E[T(n)] = O(n)$$

جایگذاری

$$E[T(n)] \leq cn$$

حکم

$$\begin{aligned} E[T(n)] &\leq \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} ck + an \\ &= \frac{2c}{n} \left( \sum_{k=1}^{n-1} k - \sum_{k=1}^{\lfloor n/2 \rfloor - 1} k \right) + an \end{aligned}$$

برای  $n$  های کوچکتر از مقداری

$$T(n) = O(1)$$

# تحليل زمني RANDOMIZED-SELECT

$$E[T(n)] = \frac{2c}{n} \left( \sum_{k=1}^{n-1} k - \sum_{k=1}^{\lfloor n/2 \rfloor - 1} k \right) + an \quad E[T(n)] \leq cn \quad \boxed{\text{حکم}}$$

$$= \frac{2c}{n} \left( \frac{(n-1)n}{2} - \frac{(\lfloor n/2 \rfloor - 1) \lfloor n/2 \rfloor}{2} \right) + an$$

$$\leq \frac{2c}{n} \left( \frac{(n-1)n}{2} - \frac{(n/2 - 2)(n/2 - 1)}{2} \right) + an$$

$$= c \left( \frac{3n}{4} + \frac{1}{2} - \frac{2}{n} \right) + an$$

$$\leq \frac{3cn}{4} + \frac{c}{2} + an$$

$$= cn - \left( \frac{cn}{4} - \frac{c}{2} - an \right)$$

$$n \geq \frac{c/2}{c/4 - a} = \frac{2c}{c - 4a}$$

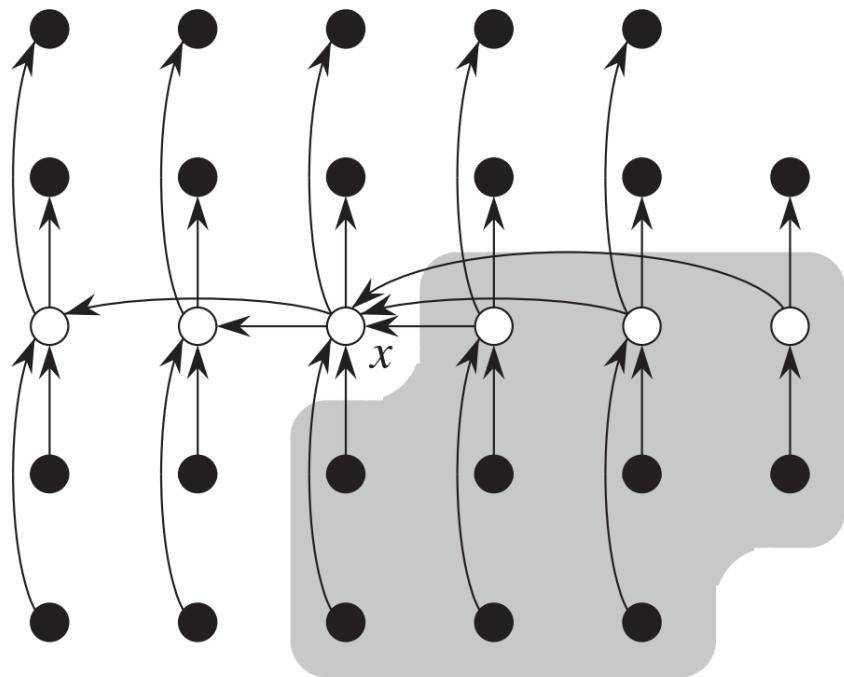
$$T(n) = O(1) \text{ for } n < 2c/(c - 4a)$$

$$E[T(n)] = O(n)$$

# مسئله انتخاب با بدترین زمان اجرای خطی

- ورودی: مجموعه  $A$  با  $n$  عدد یکتا و یک عدد طبیعی  $i$  به شرط  $1 \leq i \leq n$
- خروجی: عنصر  $x \in A$  به شرطی که  $x$  دقیقا از  $i - 1$  عنصر دیگر مجموعه  $A$  بزرگتر باشد
- ایده اصلی: مشابه روش قبل (recursive partitioning) اما با ضمانت تقسیم بندی خوب!
- مراحل تابع SELECT
- مرحله ۱: تقسیم ورودی به  $\lfloor n/5 \rfloor$  گروه ۵ عنصری و حداکثر یک گروه با تعداد عناصر ۵ یا کمتر
- مرحله ۲: یافتن میانه در هر  $\lfloor n/5 \rfloor$  گروه – **مرتب‌سازی درجی و انتخاب عنصر میانه**
- مرحله ۳: با تابع SELECT بصورت بازگشتی میانه‌ی میانه‌ها را از  $\lfloor n/5 \rfloor$  میانه انتخاب میکنیم
- مرحله ۴: Partition را با استفاده از میانه‌ی میانه‌ها  $x$  انجام بده ( $x$  برابر  $k$  امین عنصر)
- مرحله ۵: اگر  $i = k$  پس  $x$  جواب است.  
 اگر  $i < k$  در سمت چپ (پایین) و  
 اگر  $i > k$  در سمت راست (بالا) دنبال عدد بگرد

# مسئله انتخاب با بدترین زمان اجرای خطی



- دایره: اعداد داخل آرایه
- ستون: گروه‌های ۵ عنصری
- دایره سفید: میانه هر گروه
- $x$ : میانمای میانه‌ها
- پیکان: از بزرگ‌تر به کوچک‌تر
- ناحیه خاکستری: اعداد بزرگ‌تر از  $x$

• حداقل نیمی از میانه‌ها بزرگ‌تر یا مساوی میانه‌ها  $x$  هستند

• حداقل نیمی از  $\lceil n/5 \rceil$  گروه حداقل ۳ عنصر بزرگ‌تر از  $x$  دارند

• گروه کوچک و گروه خود  $x$  را استثنا می‌کنیم (حداقل)

• بالعکس آن هم صادق است! حداقل همین تعداد کوچک‌تر داریم

$$3 \left( \left\lceil \frac{1}{2} \left\lceil \frac{n}{5} \right\rceil \right\rceil - 2 \right) \geq \frac{3n}{10} - 6$$

# مسئله انتخاب با بدترین زمان اجرای خطی

- مرحله ۱: تقسیم ورودی به  $\lceil n/5 \rceil$  گروه متشکل از ۵ عنصر و حداکثر یک گروه مابقی اعداد  $O(n)$
- مرحله ۲: یافتن میانه در هر  $\lceil n/5 \rceil$  گروه – مرتب‌سازی درجی و انتخاب عنصر میانه  $O(n)$
- مرحله ۳: با تابع SELECT بصورت بازگشتی میانه‌ی میانه‌ها را از  $\lceil n/5 \rceil$  میانه انتخاب میکنیم  $T(\lceil n/5 \rceil)$
- مرحله ۴: Partition را با استفاده از میانه‌ی میانه‌ها  $x$  انجام بده ( $x$  برابر  $k$  امین عنصر)  $O(n)$
- مرحله ۵: اگر  $i = k$  پس  $x$  جواب است. اگر  $i = k$  در سمت پایین و  $i = k$  در سمت بالا دنبال عدد بگرد  $T(7n/10 + 6)$

$$T(n) = T(\lceil n/5 \rceil) + T(7n/10 + 6) + O(n)$$

# مسئله انتخاب با بدترین زمان اجرای خطی

$$T(n) = T(\lceil n/5 \rceil) + T(7n/10 + 6) + O(n)$$

$$T(n) = O(n) \quad \text{جایگذاری}$$

$$\begin{aligned} T(n) &\leq c \lceil n/5 \rceil + c(7n/10 + 6) + an \\ &\leq cn/5 + c + 7cn/10 + 6c + an \\ &= 9cn/10 + 7c + an \\ &= cn + (-cn/10 + 7c + an), \end{aligned}$$

$$T(n) \leq cn \quad \text{حکم}$$

$$n \geq 140$$

$$n/(n - 70) \leq 2$$

$$c \geq 20a$$

$$\boxed{-cn/10 + 7c + an \leq 0} \quad c \geq 10a(n/(n - 70))$$

$$T(n) \leq \begin{cases} O(1) & \text{if } n < 140 \\ T(\lceil n/5 \rceil) + T(7n/10 + 6) + O(n) & \text{if } n \geq 140 \end{cases}$$